Format Roadmap — Ketentuan Penyusunan Roadmap Bahasa Pemrograman

> **Dokumen ini adalah KETENTUAN untuk AI.** Baca file ini sebelum membuat atau merevisi sebuah roadmap.
> **Hasil akhir yang diharapkan:** satu file `Roadmap/<Kategori>/Roadmap <Topik>.md` yang siap dipakai belajar.
>
> **Cakupan dokumen ini: BAHASA PEMROGRAMAN** (JavaScript, TypeScript, PHP, Python, Go, Rust, dsb.).
>
> - Untuk **framework** (Laravel, React, NestJS, Vue, Godot) → `Format Roadmap Framework.md`.
> - Untuk **tool/teknologi non-bahasa** (Git, Docker, SQL/MySQL, HTML/CSS, terminal) → `Format Roadmap Tool & Teknologi.md`.
> - Dokumen ini tetap menjadi acuan **struktur umum** yang dipakai kedua dokumen tersebut.

---

0. Tujuan & Kapan Dipakai

Dokumen ini dipakai saat AI diminta **membuat roadmap baru** untuk satu **bahasa pemrograman**. Menerima bahasa atau teknologi yang belum jelas kategorinya, dan ikut dipakai framework/tool yang tidak punya ketentuan khusus.

Tujuan roadmap:

1. Menjadi **peta belajar** yang urut, lengkap, dan tidak membingungkan.
2. Memastikan **21 materi fundamental** selalu tercakup, disesuaikan dengan bahasa/teknologi yang dipelajari.
3. Memastikan **fitur khas** bahasa/teknologi itu ikut dipelajari (bukan cuma konsep umum).
4. Menjadi **induk** dari file materi (lihat `Format Penjelasan Materi.md`) dan file project (lihat `Format Pembuatan Project.md`).

> **Aturan penempatan file:** roadmap disimpan di folder kategori yang sesuai, contoh:
>
> - `Roadmap/Web/Roadmap HTML.md`
> - `Roadmap/Backend/Roadmap PHP.md`
> - `Roadmap/Game Dev/Roadmap Godot.md`

> - `Roadmap/Proyek/Roadmap Project PHP.md` (khusus roadmap project, lihat format terpisah)

---

1. Prinsip Wajib (tidak boleh dilanggar)

Sebuah roadmap HARUS memenuhi 7 prinsip ini:

#	Prinsip	Artinya
1	**Terstruktur**	Ada level/fase yang jelas, tiap level punya tema, bagian, dan checkpoint. Tidak ada materi yang "nyempil" tanpa alasan.
2	**Lengkap**	Menyentuh seluruh 21 materi fundamental (Bagian 3) + fitur khusus bahasa (Bagian 5) + materi ekosistem (tooling, git, testing).
3	**Benang Merah**	Setiap bagian menjelaskan kaitannya dengan bagian sebelumnya dan sesudahnya. Pembaca tidak pernah bertanya "kenapa materi ini muncul sekarang?".
4	**Urut secara Dependency**	Materi A harus dikuasai sebelum materi B. Tidak ada materi yang membutuhkan konsep yang belum diajarkan.
5	**Satu Project, Tumbuh Bersama**	Semua level membangun SATU aplikasi yang sama, dari sederhana ke kompleks (mis. sistem perpustakaan).
6	**Mengapa sebelum Bagaimana**	Setiap bagian diawali alasan "mengapa ini penting" dan "mengapa sekarang".
7	**Output Nyata**	Tiap level punya hasil yang bisa dilihat/dijalankan + checklist pemahaman.

---

2. Struktur Baku File Roadmap

Ikuti urutan heading berikut **persis**. Ini format yang sudah dipakai di `Roadmap/Backend/Roadmap PHP.md`.

2.1 Kepala file

```
# 🗺️ Roadmap <Topik>: <Subjudul Menarik>

## Filosofi Roadmap Ini
```

```
> **"<kutipan singkat yang menjelaskan sudut pandang/urgensi topik>"** — satu paragraf alasan, bukan sekadar hafal sintaks.

### Prinsip Desain

- **Satu Project, Tumbuh Bersama**: <jelaskan aplikasi apa yang dibangun, dari X → Y → Z>
- **<Prinsip 2>**: <penjelasan singkat>
- **Benang Merah Eksplisit**: setiap langkah terhubung ke langkah sebelum dan sesudahnya
- **<Prinsip 4>**: <penjelasan singkat>

### Prasyarat Sebelum Memulai
```

Sebelum roadmap ini, pastikan sudah memahami:
├── <prasyarat 1>
├── <prasyarat 2>
└──

2.2 Gambaran besar

```
## 📋 Gambaran Besar — Apa yang Akan Dibangun
```

Level 1: "" —
↓ (enhance)
Level 2: + →
↓ (enhance)
...
Level N: + →

2.3 Tiap level

Setiap level WAJIB memakai pola ini:

```
## 🟢 LEVEL N: <TEMA> (Minggu a–b)
```

```
> **Tema**: _<satu kalimat tujuan level>_
> **Benang Merah**: <Bagian sebelumnya> → <bagian ini> → <bagian sesudahnya>
> **Output**: <hasil nyata yang bisa dijalankan setelah level ini>

---

### A. <Nama Bagian>

> 💡 **Mengapa dimulai di sini?** <alasan, kaitkan dengan materi sebelum/sesudah>
```

Benang Merah Bagian A:
→
→
<hasil/poin berikutnya>

```
1. `[[<Nomor>. <Judul Materi>]]`

    - <poin penjelasan>
    - <poin penjelasan, sertakan kode singkat bila perlu>
    - _Langkah konkret_: <aksi nyata yang harus dilakukan pembaca>

2. `[[<Nomor>. <Judul Materi>]]`
    ...
```

> Emoji level: 🟢 Level dasar, 🔵 Level menengah, 🟡/🟠/🔴 Level lanjut. Boleh disesuaikan, tapi konsisten antar-file.

2.4 Checkpoint tiap level

Setiap level **harus** diakhiri checkpoint:

```
### 🏁 Checkpoint Level N
```

✅ Checklist sebelum lanjut ke Level N+1:

PEMAHAMAN:
├── Bisa jelaskan <konsep 1>
├── Bisa jelaskan <konsep 2>
└── Bisa jelaskan <konsep 3>

PROYEK:
├── <file/fitur yang harus selesai>
└── <file/fitur lain>

KEBIASAAN (HABIT):
├──
└──

Commit: <conventional commit message, mis. feat: ...>

---

3. Kurikulum Inti — 21 Materi Fundamental

Ini daftar **wajib**. Roadmap apa pun untuk bahasa pemrograman HARUS memetakan ke-21 materi ini (boleh digabung/dipecah, tapi semuanya harus kebahas). Adaptasi istilah dan sintaksnya ke bahasa yang dipelajari (lihat Bagian 4).

No	Materi	Yang harus dibahas	Boleh di level
1	**Variabel**	deklarasi, penamaan, assignment, konstanta	Dasar
2	**Tipe Data**	primitif, null, kompleks, tipe statis vs dinamis	Dasar
3	**Operator**	aritmatika, logika, perbandingan, assignment, khusus bahasa	Dasar
4	**Condition**	if/else, switch/match, ternary	Dasar
5	**Looping**	for, while, foreach/for-of, kontrol loop	Dasar
6	**Array**	indexed, associative, multidimensi, operasi dasar	Dasar
7	**Function**	definisi, parameter, default, return, type hint	Dasar
8	**OOP**	class, object, inheritance, encapsulation, polymorphism, interface/abstraksi	Menengah
9	**Input/Output (I/O)**	baca input, tampilkan output, standard I/O	Dasar
10	**String Manipulation**	slice, split, join, replace, trim, concat, format	Dasar-Menengah

No	Materi	Yang harus dibahas	Boleh di level
11	**Type Casting / Konversi Tipe**	konversi eksplisit & implisit, type juggling	Dasar
12	**Scope & Lifetime**	global/local/block scope, closure, hoisting/LEGB	Menengah
13	**Error / Exception Handling**	try-catch/finally, throw, custom error, error values	Menengah
14	**Data Structures Dasar**	Map/Hash, Set, Stack, Queue di luar array	Menengah
15	**Recursion**	fungsi rekursif, base case, tracing	Menengah
16	**Reference vs Value**	pass by value/reference, copy vs share	Menengah
17	**Modularitas (Import/Export)**	pecah file/module, package manager, autoload	Menengah
18	**File Handling**	baca/tulis/manipulasi file	Menengah
19	**Asynchronous Programming**	callback, promise/future, async-await, event loop/concurrency	Lanjut
20	**Functional Programming Basics**	higher-order function, map/filter/reduce, pure function	Menengah
21	**Debugging**	baca error, breakpoint, logging, debugger	Semua level

> **Catatan:** untuk teknologi yang bukan bahasa umum (HTML, CSS, SQL, Git), sesuaikan: ambil yang relevan saja dan jelaskan kenapa materi tertentu tidak berlaku.

Di luar 21 fundamental, roadmap full-stack juga sebaiknya memasukkan (opsional per topik): Web Fundamentals (HTML/CSS/HTTP), Frontend, Backend, Database, DevOps, Security, Testing, Arsitektur. Rujuk `Roadmap/Materi Wajib.md` untuk daftar lengkapnya.

---

4. Pemetaan 21 Materi → Sintaks per Bahasa

Saat menulis roadmap bahasa tertentu, isi kolom terkait. Tabel ini jadi "kamus konsep → sintaks".

No	Materi	JavaScript / TypeScript	PHP	R
1	Variabel	`let` / `const` / `var`	`$nama = ...`	

No	Materi	JavaScript / TypeScript	PHP	P
2	Tipe Data	string, number, boolean, null, undefined, object, symbol, bigint	string, int, float, bool, array, object, null	s li
3	Operator	`===`, `!==`, `??`, `?.`, `**`	`.=`, `<=>`, `??`, `??=`	
4	Condition	if/else, switch, ternary	if/elseif/else, switch, `match` (8.0)	if
5	Looping	for, for-of, for-in, while, do-while	for, foreach, while, do-while	f w
6	Array	`[]`, `.map/.filter/.reduce`	`[]` indexed & associative	li
7	Function	`function`, arrow `() =>`	`function`, arrow `fn()`	
8	OOP	`class` (prototype-based)	class, interface, trait	c r
9	I/O	`console`, DOM, `fetch`	`echo`, `$_GET/$_POST`, `fgets`	
10	String	template literal, `.slice/.split/.replace`	`substr/explode/str_replace/sprintf`	s f
11	Type Casting	`Number()`, `String()`, `parseInt`	`(int)`, `intval`	
12	Scope	function/block scope, hoisting, closure	function scope (`global`), `static`	L
13	Error Handling	try/catch/finally, `throw`, `Error`	try/catch/finally, `Throwable`	t
14	Data Structures	Object, `Map`, `Set`	array, `SplStack`, `SplQueue`	c
15	Recursion	fungsi rekursif	fungsi rekursif	f
16	Reference vs Value	primitif by value, object/array by reference	array copy-on-write, object by reference	s (
17	Modularitas	ESM `import/export`, CJS `require`	`require/include`, namespace, Composer (PSR-4)	
18	File Handling	Node `fs`, browser File API	`file_get_contents`, `fopen`	

No	Materi	JavaScript / TypeScript	PHP	F
19	Async	Promise, async/await, event loop	sinkron per-request (async via Swoole/ReactPHP)	
20	Functional	map/filter/reduce, pure fn, HOF	array_map/filter/reduce, `fn`	r
21	Debugging	DevTools, `console`, `debugger`	`var_dump()`, Xdebug	

---

5. Fitur Khusus per Bahasa/Teknologi (wajib dibahas)

Selain 21 fundamental, setiap bahasa/teknologi punya **ciri khas** yang tidak ada di bahasa lain. Ini yang membedakan "bisa bahasa" dan "paham bahasa". Masukkan sebagai bagian tersendiri di level yang tepat.

Bahasa Pemrograman

- **JavaScript**: closure, prototype chain, `this` binding, hoisting, event loop, Promise/async-await, destructuring, spread/rest, modul ESM vs CJS, `Proxy` / `Reflect`, generator, WeakMap/WeakSet, DOM & Web API.
- **TypeScript**: static typing, `interface` vs `type`, generics, union/intersection, type narrowing & guards, decorator, enum, utility types (`Partial`, `Pick`, `Omit`, `Record`), mapped & conditional types, declaration files (`.d.ts`), tsconfig.
- **PHP**: variabel `$`, superglobal (`$_GET/$_POST/$_SESSION/$_FILES`), type juggling & `===`, array as map, Composer + PSR-4 autoload, trait, namespace, closure `use`, magic methods (`__construct`, `__get`), `match` & attribute (PHP 8), PDO & prepared statement, session & cookie.
- **Python**: indentasi sebagai blok, dynamic typing + type hints, list/dict/set comprehension, decorator, generator (`yield`), context manager (`with`), `*args/**kwargs`, slicing, virtualenv/pip, GIL, duck typing.
- **Go**: goroutine & channel (concurrency), error sebagai nilai (bukan exception), interface implicit, struct & method, `defer/panic/recover`, slice vs array, `go mod`, kompilasi ke binary tunggal, tanpa class/inheritance.
- **Java**: JVM, strongly typed, generics, lambda & Stream API, garbage collector, checked exception, annotation, Maven/Gradle.
- **C# / .NET**: LINQ, async/await, property, delegate/event, record, nullable reference types, GC.
- **Rust**: ownership, borrowing, lifetime, borrow checker, trait, `enum` + pattern matching, `Result` / `Option`, tanpa GC, Cargo, macro.

- **C / C++**: pointer, manajemen memori manual, struct, preprocessor, RAII & STL (C++), template.
- **Kotlin**: null safety, coroutine, data class, extension function, interop JVM.
- **Ruby**: "semuanya object", block/proc/lambda, metaprogramming, gem, symbol, duck typing.

Teknologi Web & Framework

- **HTML/CSS** (bukan bahasa pemrograman): struktur & semantik dokumen, box model, flexbox/grid, responsive, specificity & cascade.
- **SQL/MySQL**: deklaratif, `JOIN`, `INDEX`, transaksi & ACID, normalisasi, `GROUP BY`, subquery.
- **Node.js**: event loop, `npm`, module system, stream, `Buffer`, `cluster`, environment variable.
- **React**: komponen, JSX, hooks (`useState/useEffect`), props vs state, context, reconciliation/virtual DOM, client-side routing.
- **Vue**: reactivity (ref/reactive), directive (`v-if/v-for/v-model`), SFC, Composition API, computed/watch.
- **Laravel**: Eloquent ORM, Blade, Artisan CLI, middleware, migration & seeder, service container & DI, validation, queue & event.
- **NestJS**: module, decorator, dependency injection, guard/interceptor/pipe/filter, DTO & validation, Prisma/TypeORM.
- **Tailwind CSS**: utility-first, responsive variant, `@apply`, konfigurasi & purge.
- **Bootstrap**: grid system, komponen, utility.

> Saat menulis roadmap, buat satu bagian/level khusus bertema ": fitur khas & hal yang membedakannya dari bahasa lain".

---

6. Aturan Penomoran, Link, dan Konsistensi

1. **Penomoran materi** berurutan global (1, 2, 3, …) lintas level — bukan reset tiap level. Ini nomor yang dipakai untuk menautkan ke file materi.
2. **Judul materi** seragam: `<Nomor>. <Judul> — <Subjudul Penjelas>`.
   - Contoh: `6. Variabel — Wadah Data dengan Tanda Dollar`.
3. **Link ke materi** memakai wiki-link: `` `[[6. Variabel — Wadah Data dengan Tanda Dollar]]` ``.
   - Di dalam file **Roadmap Project**, boleh pakai path lengkap: `[[Belajar/IT/Pemrograman/Project/Project PHP Native/Level 1 - Fondasi`

`PHP/1. ...]]`.

4. **Setiap nomor di roadmap harus punya file materinya** (dibuat memakai `Format Penjelasan Materi.md`). Jika belum ada, tandai eksplisit "belum dibuat".
5. **Nama level** seragam: `Level N – <Nama>`; bagian: `A.`, `B.`, `C.`, ...
6. **Konsisten antar-roadmap**: struktur, emoji, dan istilah (Tema/Benang Merah/Output/Checkpoint) selalu sama.

---

7. Level, Durasi, dan Output

- Setiap level punya **estimasi minggu** (mis. `(Minggu 1–4)`).
- Jumlah level wajar: 5-8 level. Lebih dari itu, pecah jadi beberapa roadmap.
- Setiap level harus punya **Output** yang konkret (fitur/file yang jalan), bukan sekadar "paham teori".
- Setiap level diakhiri **Checkpoint** dengan 3 blok: PEMAHAMAN, PROYEK, KEBIASAAN + saran pesan commit.

---

8. Template Roadmap (copy-paste)

```
# 🗺️ Roadmap <Topik>: <Subjudul>

## Filosofi Roadmap Ini

> **"<kutipan>"** – <alasan singkat>

### Prinsip Desain

- **Satu Project, Tumbuh Bersama**: <deskripsi>
- **<Prinsip 2>**: <deskripsi>
- **Benang Merah Eksplisit**: setiap langkah terhubung ke langkah sebelum dan sesudahnya
- **<Prinsip 4>**: <deskripsi>

### Prasyarat Sebelum Memulai
```

Sebelum roadmap ini, pastikan sudah memahami:
├──

```
---

## 📋 Gambaran Besar — Apa yang Akan Dibangun
```

Level 1: ...
↓ (enhance)
Level N: ...

```
---

## 🟢 LEVEL 1: <TEMA> (Minggu 1–x)

> **Tema**: _<...>_
> **Benang Merah**: <...>
> **Output**: <...>

---

### A. <Bagian>

> 💡 **Mengapa dimulai di sini?** <...>
```

Benang Merah Bagian A:
<...> → <...> → <...>

```
1. `[[1. <Judul> — <Subjudul>]]`

    - <poin>
    - _Langkah konkret_: <aksi>

---

### 🧗 Checkpoint Level 1
```

✅ Checklist sebelum lanjut ke Level 2:
...

---

9. Checklist Kualitas Roadmap

Sebelum dianggap selesai, cek:

- [ ] Ada **Filosofi**, **Prinsip Desain**, dan **Prasyarat**.
- [ ] Ada **Gambaran Besar** (alur level).
- [ ] Semua level punya **Tema**, **Benang Merah**, dan **Output**.
- [ ] Setiap bagian punya alasan "**Mengapa dimulai di sini?**" + alur benang merah.
- [ ] **21 materi fundamental** semuanya tercakup (atau dijelaskan kenapa tidak berlaku).
- [ ] **Fitur khusus bahasa** dibahas.
- [ ] Setiap level punya **Checkpoint** (pemahaman + proyek + kebiasaan + commit).
- [ ] Penomoran berurutan global & setiap nomor punya link `[[...]]`.
- [ ] Urutan materi bebas dari dependency yang terbalik.
- [ ] Satu project yang sama tumbuh dari level pertama sampai terakhir.

File terkait:

- `Format Penjelasan Materi.md` — cara menulis isi tiap poin roadmap.
- `Format Pembuatan Project.md` — cara menyusun roadmap project.
- `Format Penjelasan Materi Project.md` — cara menulis materi project.
- `Roadmap/Materi Wajib.md` — daftar materi pendukung (web, backend, DB, devops, security, testing).